

CS305 Fall 2025 Setup Tutorial

This tutorial is provided to help you understand the code implementation of the Project.

We strongly recommend reading this Setup Tutorial after you have a comprehensive understanding of the project content, and you can read this in conjunction with the code.

The command-line *Example Process* section is **just to help you understand the project's operation. In our testing**, we will prepare the relevant fragment data, and directly execute some commands; you can also refer to the test scripts to **write your own test cases**.

CS305 Fall 2025 Setup Tutorial

1. Project Code Framework
 - 1.1 Directory Structure
 - 1.2 Directory Overview
 - 1.3 Code Key Files
 - 1.3.1 Creating Packets in Python
 - 1.3.2 Default `.gitignore` for the Repository
2. Transfer Protocol Details
 - 2.1 P2P Key File Definitions
 - 2.2 Setup Network Simulator
 - 2.3 Setup Peers
 - 2.3.1 Prepare Chunk Files
 - 2.3.2 Peer Configuration
 - 2.4 Listening
 - 2.5 Handshaking
3. Example Process
 - 3.1 Example Overview
 - 3.2 Prepare chunk files
 - 3.3 Run Example With Simulator
 - 3.3.1 Start the Simulator
 - 3.3.2 Start peers (demo_receiver and demo_sender)
 - 3.4 Invoke downloading in demo_receiver
4. Run `pytest` Script
5. Linux Setup
 - 5.1 WSL
 - 5.1.1 Useful references
 - 5.2 Virtual Machine
 - 5.2.1 Useful references
 - 5.3 Docker
 - 5.3.1 Useful references

1. Project Code Framework

1.1 Directory Structure

The structure of all provided files is as follows:

```
1 | sustech-cs305-f25-project-starter/  
2 | └─ example/  
3 |   └─ __init__.py  
4 |   └─ demo_receiver.py
```

```

5 | | └─ demo_sender.py
6 | | └─ ex_file.tar
7 | | └─ ex_nodes_map
8 | | └─ ex_topo.map
9 | └─ src/
10 | | └─ __init__.py
11 | | └─ peer.py
12 | └─ test/
13 | | └─ checkersocket.py
14 | | └─ grader.py
15 | | └─ test_01_basic_handshaking.py
16 | | └─ tmp1/
17 | └─ utils/
18 | | └─ __init__.py
19 | | └─ hupsim.pl
20 | | └─ make_data.py
21 | | └─ nodes.map
22 | | └─ peer_context.py
23 | | └─ simsocket.py
24 | | └─ topo.map

```

1.2 Directory Overview

The four main directories serve the following purposes:

- **example/**: Contains a simple, runnable stop-and-wait implementation that demonstrates the basic usage of the provided framework.
- **src/**: **The directory you need to submit.** All of your implementation code must be written within this folder, primarily by completing the `peer.py` main file.
- **test/**: Includes public test scripts to help you verify the correctness of your implementation. Additional tests will be released at different times.
- **utils/**: Provides various supporting modules and scripts required for the project.

1.3 Code Key Files

Below is a description of the most important files in the project code framework:

- **src/peer.py**: The main file for your implementation. You are expected to complete this file to meet the project requirements.
 - **Note:** You must use the provided `simssocket` for all network operations; normal sockets are not permitted. We allow you to use better code design that conforms to best practices. Ensure that your program's entry point is `peer.py`, which can parse the given command-line arguments.
- **utils/simssocket.py**: Provides the `Simsocket` class, a modified socket that can run with or without the network simulator. **Do not modify this file.**
- **utils/hupsim.pl**: A network simulator written in Perl that can emulate routing, queuing, congestion, and packet loss.
- **utils/make_data.py**: A Python script used to split files into chunks and generate their corresponding chunk hashes.

- `utils/nodes.map`: A file that lists all peers in the network and their corresponding addresses for the simulator.
- `utils/topo.map`: Defines the network topology for the simulator, specifying links between nodes.
- `utils/peer_context.py`: A utility module for parsing command-line arguments. It parses these arguments into a "P2P context" object, which contains all the peer's configuration settings. You do not need to modify this file.
- `test/grader.py`: Provides the grading session for running automated tests.
- `test/test_01_basic_handshaking.py`: An example test script that can be invoked by `pytest` to check your handshaking logic. We initially released **4 basic function-related test scripts**; this file is an example.
- `example/demo_sender.py`: An example implementation of a sender using a simple stop-and-wait protocol.
- `example/demo_receiver.py`: An example implementation of a receiver that reads user input and handles the download process.

1.3.1 Creating Packets in Python

Use `struct`. Refer to our example code and the [Python struct document](#)

1.3.2 Default `.gitignore` for the Repository

In addition to the common Python-related ignores, note our extra default ignore file. If you want to include certain files in version control, remove them from the `.gitignore` file in the root directory.

```

1  # Download Result
2  download_result.fragment
3  result*.fragment
4
5  # Root chunkhash for example
6  /master.chunkhash
7
8  # Analysis plot file
9  concurrency_analysis.png
10
11 # logs
12 logs/

```

2. Transfer Protocol Details

2.1 P2P Key File Definitions

`*.xxx` denotes all files with suffix `.xxx`

The **chunkdata** of a chunk is its 512KiB data bytes.

The **chunkhash** of a chunk is a 20-byte SHA-1 hash value of its chunkdata.

- `\*.fragment`: serialized dictionary of the form `chunkhash: chunkdata`. It is an input file to a peer, and it will be automatically loaded into a dictionary when running a peer. See the example peer [`example/demo_sender.py`, `example/demo_receiver.py`] for detail. In addition, once you **complete** a `DOWNLOAD` task, you should store your downloaded chunks in a dictionary of form `chunkhash: chunkdata`, and serialize it to the given file name.
- `\*.chunkhash`: Files that only contain chunkhashes. These files appear as `master.chunkhash` that holds all chunkhashes of the file, and `downloadxxx.chunkhash` that tells a peer what to download.

2.2 Setup Network Simulator

To test your system, you will need networks that simulate realistic conditions that can have loss, delay, and many nodes causing congestion. To help you with this, we created a simple network simulator called "Spiffy" which runs completely on your local machine. The simulator is implemented by `hupsim.pl`, which creates a series of links with limited bandwidth and queue sizes between nodes specified by the file `topo.map` (this allows you to test congestion control). To run your peers on the virtual network, you need to set an environment variable `SIMULATOR` before running peers:

```
1 export SIMULATOR="<simulator_ip>:<simulator_port>"
```

And then run your simulator from another shell:

```
1 perl utils/hupsim.pl -m <topo-file> -n <nodes-file> -p <port> -v <verbose-level>
```

- `-m <topo-file>`: This is the file containing the configuration of the network that `hupsim.pl` will create. An example is given to you as `topo.map`. The ids in the file should match the ids in the `nodes.map` file. Each line defines a link in the network, and has 5 attributes: `<node1> <node2> <bw> <delay> <queue-size>`.
 - The `bw` is the bandwidth of the link in bits per second.
 - The `delay` is the delay in milliseconds.
 - The `queue-size` is in packets.
 - Your code is **not allowed** to read this file. If you need values for network characteristics like RTT, you must infer them from network behavior. You can calculate RTT using exponential averaging.
- `-n <nodes-file>`: This is the file that contains configuration information for all nodes in the network. An example is given to you as `nodes.map`.
- `-p <port>`: This is the port that `hupsim.pl` will listen to. Therefore, this port should be **DIFFERENT** from the ports used by the nodes in the network. (Note: This port should match the one you set in the `SIMULATOR` environment variable, e.g., `12345`).
- `-v <verbose-level>`: How much debugging messages you want to see from `hupsim.pl`. This should be an integer from 1-4. Higher value means more debugging output.

After running the simulator and setting environment variable correctly, your peer will automatically run on simulator as long as you are using `simsocket`.

2.3 Setup Peers

2.3.1 Prepare Chunk Files

At the very start, we need to prepare the chunk files in the network, and generate a fragment file for each peer. A `make_data.py` script is provided to handle this:

```
1 python3 -m utils.make_data <file-to-split> <fragment-file> <num-chunks>
   <indices>
```

- **<file-to-split>**: The file to be split into chunks. It can be any binary file like `*.tar` or `*.zip`. Note that a file too small will fail to generate 512KB chunks.
- **<fragment-file>**: A `*.fragment` file, a serialized dictionary of form `chunkhash: chunkdata`. The chunkhashes in it are selected from the **<indices>**.
- **<num-chunks>**: Number of chunks to keep after partition. If this value is set `3` for a **<file-to-split>** that would normally produce 4 chunks (e.g., 2563KB), it will only keep the first 3 chunks. And if it is set `5`, it will use `4` instead of `5` because `5` is out of bounds. Note that the last 3 bytes that cannot form a chunk will be discarded, all chunks will be 512KiB.
- **<indices>**: Comma-separated indices to indicate which chunks to be selected into **<fragment-file>**. For example, `"2,4,6"` means to select chunk 2, chunk 4, chunk 6. The index starts from 1 instead of 0.

2.3.2 Peer Configuration

You will need to configure each peer by telling (i) which chunks they already own and (ii) the locations (i.e., hostname and port number) of other peers. To launch a peer, several arguments should be given, the command can be in the form of:

```
1 python3 -m src.peer -p <peer-file> -c <chunk-file> -m <max-conn> -i
   <identity> -t <timeout> -v <verbose-level>
```

- **-p <peer-file>**: This field corresponds to the path to peer-file. The peer-file contains the identity of all the peers and the corresponding hostname and port. The peer then knows all the other peers in the network.
- **-c <chunk-file>**: The **<chunk-file>** is a `*.fragment` file. It is a serialized dictionary of form `chunkhash: chunkdata`. This file is generated from `make_data.py`. You do not need to load it manually, it will be loaded automatically. See the example for more information.
- **-m <max-conn>**: Max number of peers that this peer is able to send packets to. If more peers make requests to this peer, it should send back `DENIED` packet.
- **-t <timeout>**: If timeout is not set, you should estimate RTT in the network to set your timeout value. However, if it is set, you should always use this value. A pre-defined timeout value will be used in the testing part.
- **-i <identity>**: The identity (ID) of the current peer, which will help to distinguish the peers. This identity should be used by the peer to get its own location (i.e. hostname and port) from peer-file, and then bind its listening socket to that address.
- **-v <verbose-level>**: Level of verbosity. From 0 to 3.

Detailed Examples for peer setup can be found in the **Example Process** section .

2.4 Listening

Each peer will keep listening to the UDP socket and user input until termination. If the peer receives a UDP packet or user input, then it should process them respectively according to the following instructions. If the peer does not receive any packets or user input within a given time (e.g., the `select()` call times out), it should handle this situation accordingly (e.g., checking for RDT timeouts).

Listening to User Input: To download chunks, a user will input:

```
1 | DOWNLOAD <chunk-hash-file> <output-fragment-file>
```

- **<chunk-hash-file>**: A `*.chunkhash` file contains hashes of chunks to be downloaded. The peer should download all chunks listed in this file.
- **<output-fragment-file>**: Name of a `*.fragment` file. It should be a serialized dictionary that stores `chunkhash: chunkdata` in which `chunkhash` is hash in the **<chunk-hash-file>** and `chunkdata` should be the downloaded data. The serialized file name should be **<output-fragment-file>**.

Then, upon receiving such a user input, the peer will read from the file in **<chunk-hash-file>** given in the command. Afterwards, the peer will need to download the chunk data from other peers according to hash values, and assemble the downloaded chunks to a dictionary. Finally, the peer will write the serialized dictionary to the **<output-fragment-file>** and print a confirmation message (e.g., "DOWNLOAD complete").

Listening to Socket: If any UDP packet is received from the socket, the peer should handle the packet according to its type and content. Check out our code framework and examples to learn how to implement this listening loop.

2.5 Handshaking

As mentioned in our main document, upon receiving a user's `DOWNLOAD` command, the peer should gather all the requested chunk data. There will be two procedures: handshaking with other peers; chunk transferring.

We will **repeat** the handshake process here. The handshaking procedure consists of three types of messages: `WHOHAS`, `IHAVE`, and `GET`. Specifically, the peer will establish a connection with some of the other peers through a "three-way handshaking" similar to that of TCP. The "three-way handshaking" is as follows:

1. The peer sends a `WHOHAS` packet to all the peers previously known in the network in order to check which peers have the requested chunk data. A `WHOHAS` packet contains a list of chunk hashes indicating which chunks the peer needs.
2. When other peers receive a `WHOHAS` packet from this peer, they should look into which requested chunks they own respectively. They will send back to this peer with an `IHAVE` packet. Each other peer sends an `IHAVE` packet containing the hash of the requested chunks that it owns. However, if this peer is already sending to `<max-conn>` number of other peers at the time when it receives a new `WHOHAS`, it should send back a `DENIED`.

3. Once the peer receives an `IHAVE` packet from other peers, it knows which chunks each of the other peers owns. Then, the peer will choose particular peer from which it downloads each requested chunk respectively. It will send a `GET` packet containing the hash of exactly one of the requested chunks to each particular peer for chunk downloading. For example, if the peer decides to download chunk A from peer 1, then it will send a `GET` packet containing the hash of chunk A to peer 1.

Note that in step 3, the peer can send multiple `GET` packets to multiple different peers. See main documentation "*Chunk Transfer*" subsection.

3. Example Process

3.1 Example Overview

In our example, a file will be divided into 4 chunks, and peer 1 will have chunk 1 and chunk 2, while peer 2 will have chunk 3 and chunk 4. Peer 1 will be invoked to download chunk 3 from peer 2.

3.2 Prepare chunk files

We first generate chunk data and for peers:

```
1 python3 -m utils.make_data ./example/ex_file.tar ./example/data1.fragment 4
  1,2
```

This means to split `./example/ex_file.tar` into 4 512KiB chunks, and select the chunk 1 and chunk 2 to be in `./example/data1.fragment`. The `./example/data1.fragment` will be a pickle-dumped dictionary, and it will be loaded into a dictionary when running. More information about pickle can be found at [pickle doc](#).

Similarly, we can generate another chunk file using:

```
1 python3 -m utils.make_data ./example/ex_file.tar ./example/data2.fragment 4
  3,4
```

This generates `./example/data2.fragment` that contains chunk 3 and chunk 4 of the original file. This also generates a `.chunkhash` that contains all 4 chunk hashes of this file, named `master.chunkhash`. The chunk hash file will be like:

```
1 1 12e3340d8b1a692c6580c897c0e26bd1ac0eaadf
2 2 45acace8e984465459c893197e593c36daf653db
3 3 3b68110847941b84e8d05417a5b2609122a56314
4 4 4bec20891a68887eef982e9cda5d02ca8e6d4f57
```

Now create another chunk hash file to tell peer 1 which chunks to download:

```
1 sed -n '3p' master.chunkhash > example/download.chunkhash
```

`sed` is a convenient command to select lines from a file. More information about this command can be retrieved from [sed manpage](#). This command will result in a new chunk hash file `example/download.chunkhash` that only contains hash of chunk 3.

3.3 Run Example With Simulator

Now we have prepared data chunks for the example. In the following part, you will need to start multiple shells to run peers in different processes.

3.3.1 Start the Simulator

Start the simulator in your old shell:

```
1 | perl utils/hupsim.pl -m example/ex_topo.map -n example/ex_nodes_map -p 50305 -v 2
```

3.3.2 Start peers (demo_receiver and demo_sender)

Start a new shell, setup the environment variable `SIMULATOR` and run the sender:

```
1 | export SIMULATOR="127.0.0.1:50305"
2 | python3 -m example.demo_sender -p example/ex_nodes_map -c
   example/data2.fragment -m 1 -i 2 -v 3
```

Then again start another new shell, run the receiver:

```
1 | export SIMULATOR="127.0.0.1:50305"
2 | python3 -m example.demo_receiver -p example/ex_nodes_map -c
   example/data1.fragment -m 1 -i 1 -v 3
```

If you do not start the simulator (that is, do not set the `SIMULATOR` environment variable), just run `demo_sender` and `demo_receiver` and invoke downloading like the next part. You will find it much faster.

3.4 Invoke downloading in demo_receiver

Then input the following command in the receiver's shell:

```
1 | DOWNLOAD example/download.chunkhash example/test.fragment
```

This will invoke the downloading process in the receiver and save the downloaded file to `example/test.fragment`. You will see the peers running and logs will be printed to `stdout`. The downloading will finish in **about several minutes**, then it will print out:

```
1 | GOT example/test.fragment
2 | Expected chunkhash: 3b68110847941b84e8d05417a5b2609122a56314
3 | Received chunkhash: 3b68110847941b84e8d05417a5b2609122a56314
4 | Successfully received: True
5 | Congratulations! You have completed the example!
```

Now you should be able to embark on your own implementation of peers!

4. Run `pytest` Script

To run `pytest` script, try run this command in root directory:


```
1 | pytest test
```

or for specific test, run:

```
1 | pytest test/test_01_basic_handshaking.py
```

You can build your own test cases and scripts in the test folder and run them with the relevant commands.

5. Linux Setup

For students not natively using Linux, you may need to setup a Linux environment.

This section is just a simple tutorial. The following content is for reference only. If there is anything unclear, you can search for more detailed tutorials on related online resources.

5.1 WSL

You can use WSL 2 (Windows Subsystem for Linux). This is currently the simplest and most lightweight option.

Open PowerShell or the Windows Terminal as an administrator.

For example, run the following command to install Ubuntu 22.04 in one click:

```
1 | wsl --install -d Ubuntu-22.04
```

After the installation is complete, restart your computer. After restarting, find Ubuntu in the Start menu and open it. The first time you start it, you'll be asked to set a Linux username and password (this password will be needed for `sudo`).

Once you're done, you'll have a Ubuntu terminal environment.

5.1.1 Useful references

[Microsoft's Official WSL Installation Documentation](#)

[Microsoft's Official WSL Tutorials](#)

5.2 Virtual Machine

Compared to WSL, virtual machines usually allow you to use Linux distributions that include a GUI, which is easier to operate. However, its performance requirements are higher.

You'll need two things:

Virtual Machine Software (Hypervisor): This is the "player" used to create and run virtual computers.

- [Oracle VirtualBox](#): Free and open source, with powerful features.
- [VMware Workstation](#): Free for personal use, with excellent performance and a modern interface.

Operating System Image (ISO File)

- ISO file is a technology that packages the entire CD into one file, this could be the "installation disc" for Linux OS. You need to search and download the `.iso` file of the operating system you want to install.

Then, you can run ISO file in your virtual machine to install Linux OS.

5.2.1 Useful references

[Ubuntu Official Virtual Machine Tutorials](#)

5.3 Docker

For those familiar with Docker, we have provided some Docker-related code for reference in `/doc/optional_files/docker` .

5.3.1 Useful references

<https://docker-curriculum.com/>